



Université Hassan II
Faculté des sciences Ben Msik
Département : Informatique et Mathématique
Filière : MEIA Master d'Excellence en Intelligence Artificielle

TP Chapitre 2 — Pipelines agiles, versionnement et orchestration avec dlt, dbt et Dagster

Réalisé par : BOUAZZA Amina

Année universitaire : 2025/2026

Table des matières

Introduction	2
Partie 1 : Préparation du Projet et Environnement	3
Partie 2 : Données Source	5
Partie 3 : Ingestion avec dlt / Python	6
Partie 4 : Validation des Données	8
Partie 5 : Transformation avec dbt	9
Partie 6 : Tests de Qualité avec dbt	12
Partie 7 : Orchestration avec Dagster	13
Partie 8 : Intégration Continue (CI)	14
Réponses aux Questions	15
Conclusion	16

Introduction

Ce TP vise à transformer un traitement de données simple en un pipeline structuré et industrialisé en utilisant **dlt** pour l'ingestion, **dbt** pour la transformation, **Dagster** pour l'orchestration, **Git** pour le versionnement et un environnement virtuel pour la reproductibilité.

Partie 1 : Préparation du Projet et Environnement

Structure du projet

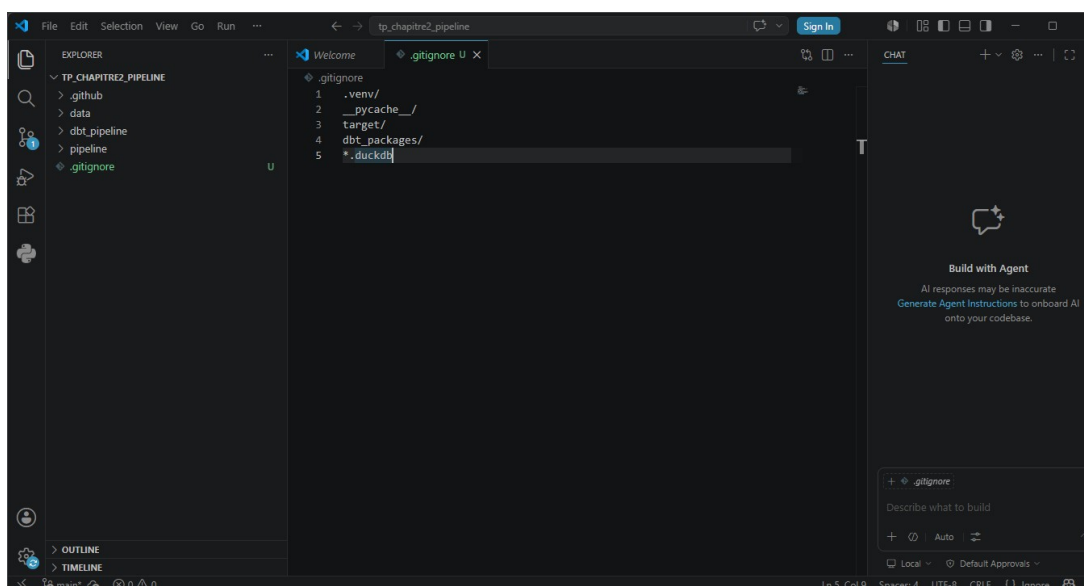
```
C:\Users\pc\Desktop\tp_chapitre2_pipeline>
C:\Users\pc\Desktop\tp_chapitre2_pipeline>tree /f
Structure du dossier
Le numéro de série du volume est 00000091 F6BC:D312
C:..
├── .github
│   └── workflows
├── data
├── dbt_pipeline
│   └── models
└── pipeline

C:\Users\pc\Desktop\tp_chapitre2_pipeline>d
```

FIGURE 1 – Structure du dossier du projet avec la commande tree.

Cette capture montre l'arborescence finale du projet avec les dossiers data, pipeline, dbt_pipeline et .github.

Fichier .gitignore



The screenshot shows the Visual Studio Code interface with the Explorer view on the left displaying the project structure. The main editor area shows the content of the .gitignore file:

```
.gitignore
1 .env/
2 __pycache__
3 target/
4 dbt_packages/
5 *.duckdb
```

The right sidebar shows the Chat panel with a 'Build with Agent' section and a 'Generate Agent Instructions' button.

FIGURE 2 – Contenu du fichier .gitignore.

Ce fichier permet d'exclure les fichiers temporaires et les bases de données locales du suivi Git.

Premier commit Git

```
C:\Users\pc\Desktop\tp_chapitre2_pipeline>git add .  
  
C:\Users\pc\Desktop\tp_chapitre2_pipeline>git commit -m "Initialisation du projet pipeline"  
[main (root-commit) 25df440] Initialisation du projet pipeline  
2 files changed, 292 insertions(+)  
create mode 100644 .gitignore  
create mode 100644 requirements.txt  
  
C:\Users\pc\Desktop\tp_chapitre2_pipeline>
```

FIGURE 3 – Initialisation du dépôt Git et premier commit.

Cette commande initialise le dépôt et effectue le commit initial du projet.

Partie 2 : Données Source

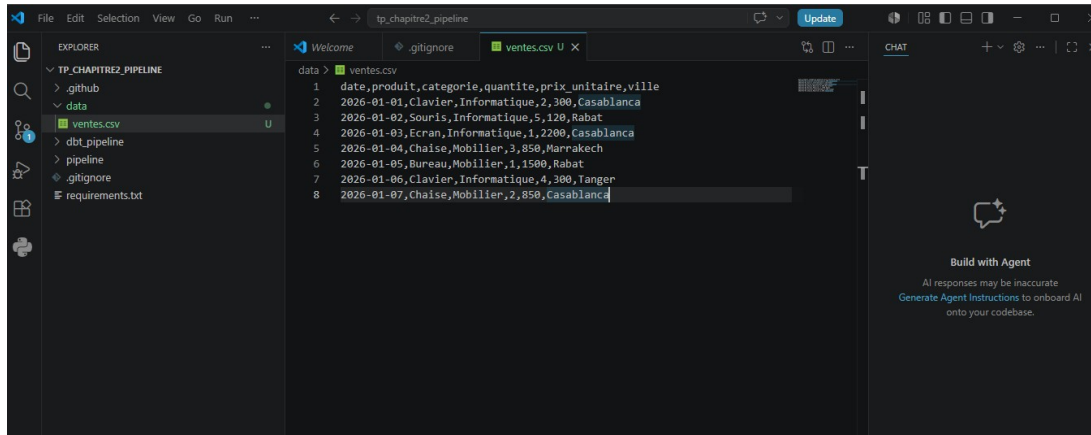


FIGURE 4 – Fichier data/ventes.csv contenant les données brutes.

Ce fichier CSV contient les ventes avec les colonnes : date, produit, catégorie, quantité, prix unitaire et ville.

Partie 3 : Ingestion avec dlt / Python

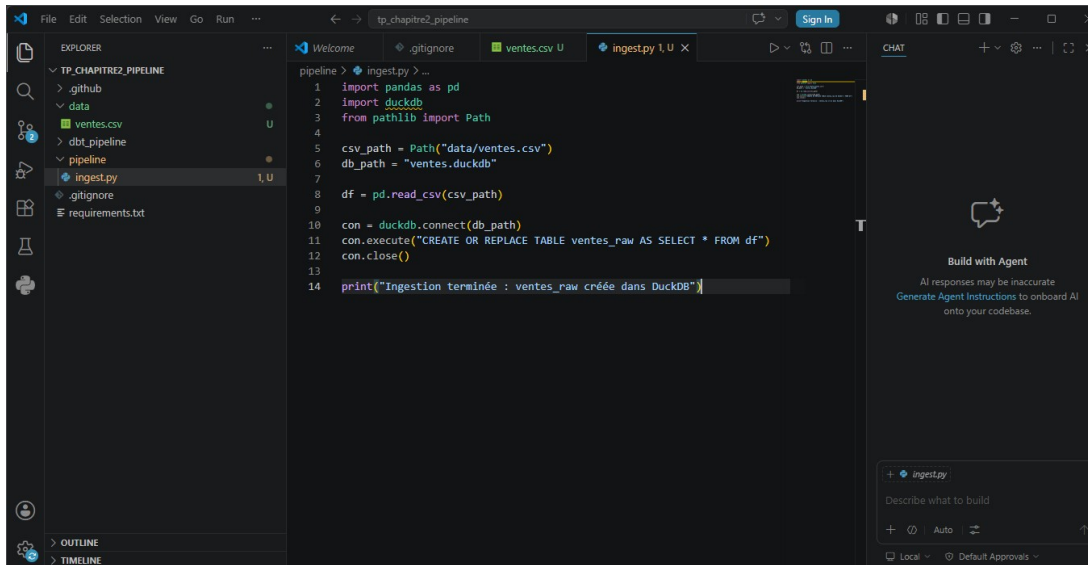


FIGURE 5 – Contenu du script pipeline/ingest.py.

Ce script lit le fichier CSV et le charge dans une table `ventes_raw` dans DuckDB.

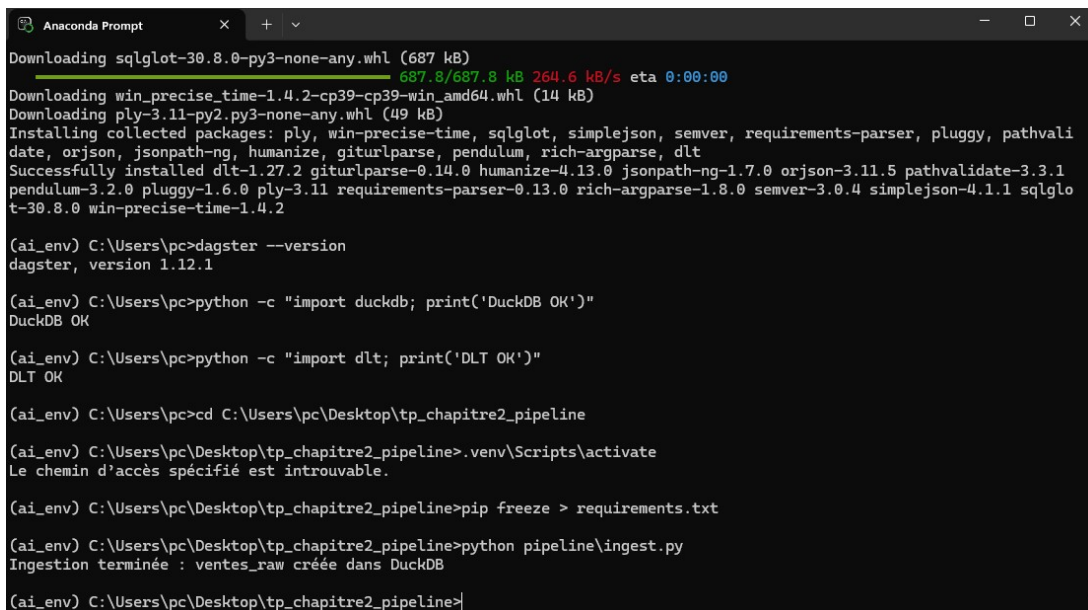


FIGURE 6 – Exécution du script d'ingestion.

L'ingestion s'est terminée avec succès et la table `ventes_raw` a été créée.

```
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>git add data\ventes.csv pipeline\ingest.py
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>git commit -m "Ajout de l'étape d'ingestion"
[main 1dd0b8d] Ajout de l'étape d'ingestion
 2 files changed, 22 insertions(+)
 create mode 100644 data/ventes.csv
 create mode 100644 pipeline/ingest.py
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>
```

FIGURE 7 – Commit Git après l'étape d'ingestion.

Partie 4 : Validation des Données

```
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>python pipeline\validate.py
Validation réussie : schéma et qualité minimale OK

(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>|
```

FIGURE 8 – Exécution du script de validation.

Le script vérifie la présence des colonnes et l'absence de valeurs nulles critiques.

```
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>git add pipeline\validate.py

(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>git commit -m "Ajout de l'étape de validation minimale"
[main 2b2224a] Ajout de l'étape de validation minimale
 1 file changed, 40 insertions(+)
 create mode 100644 pipeline/validate.py

(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>|
```

FIGURE 9 – Commit après l'ajout de l'étape de validation.

Partie 5 : Transformation avec dbt

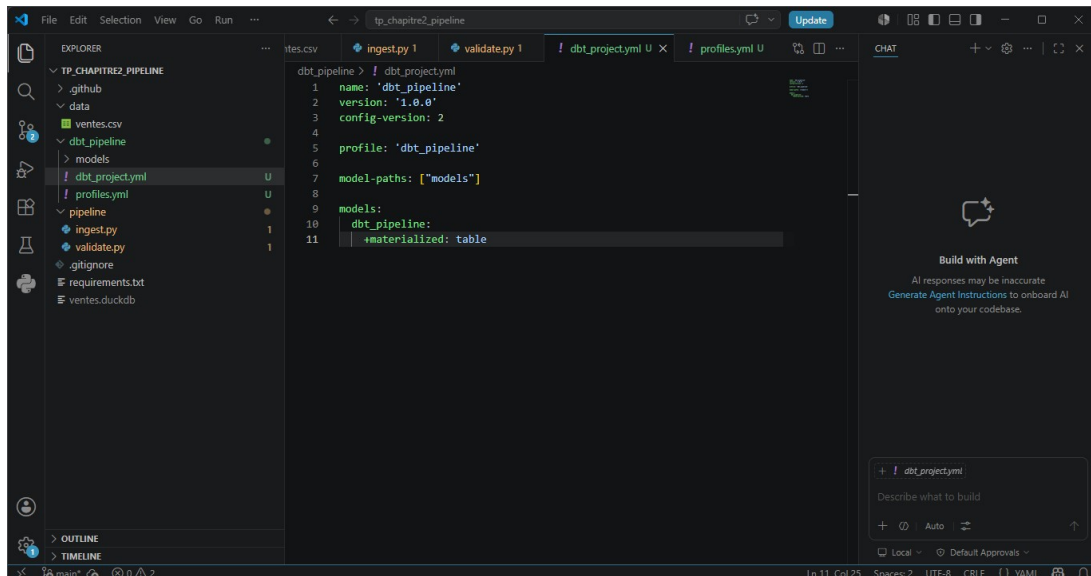


FIGURE 10 – Fichier `dbt_project.yml`.

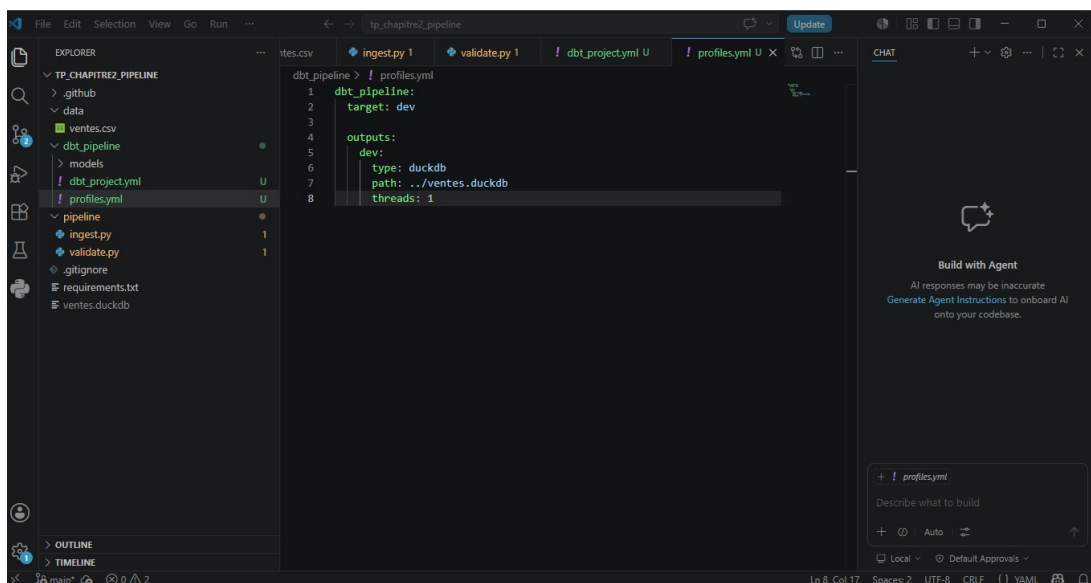


FIGURE 11 – Fichier `profiles.yml` pour la connexion à DuckDB.

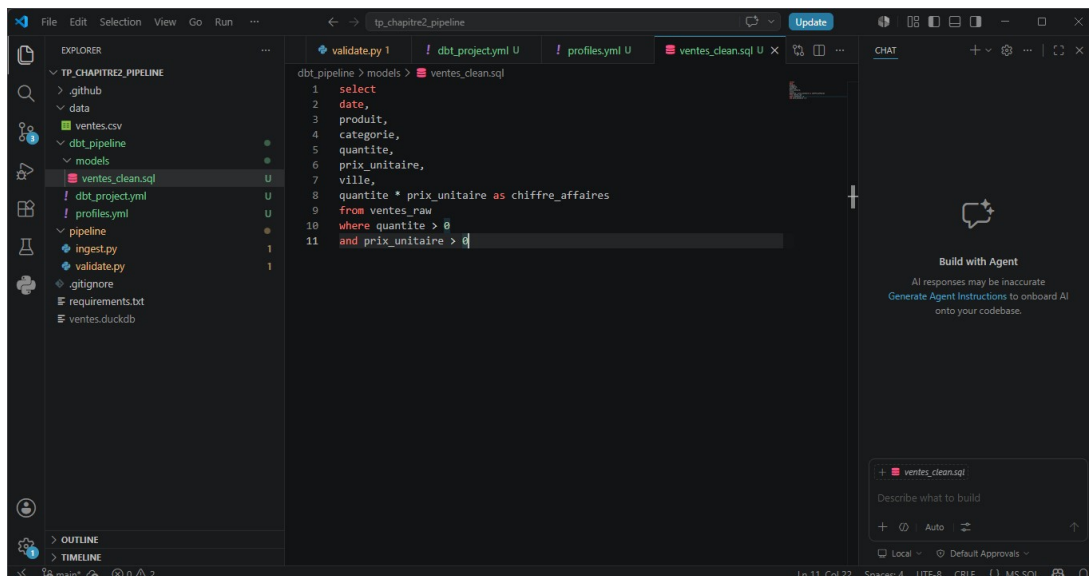


FIGURE 12 – Modèle dbt : ventes_clean.sql.

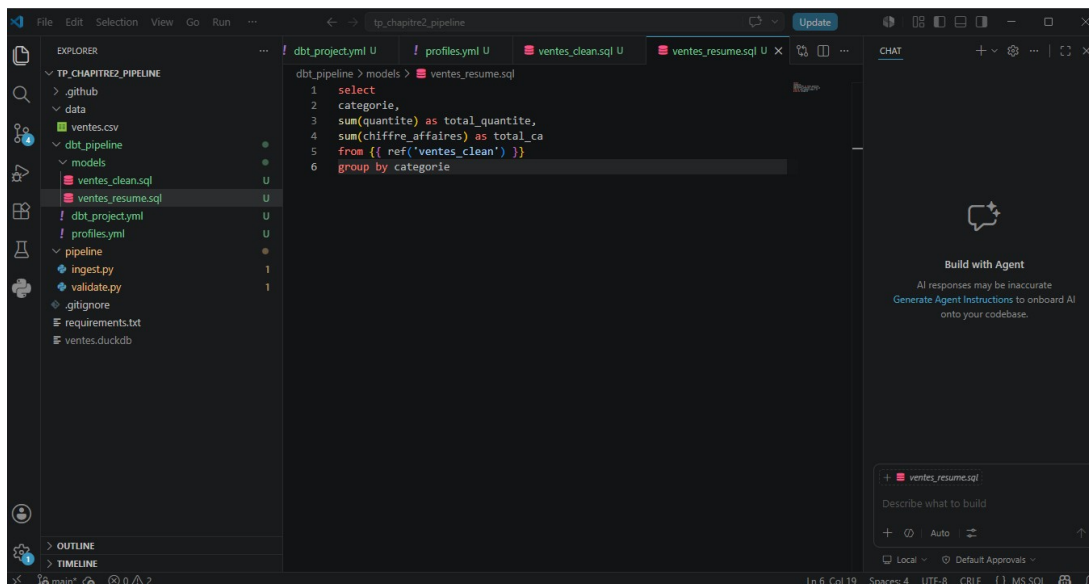


FIGURE 13 – Modèle dbt : ventes_resume.sql.

```

Anaconda Prompt
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline\dbt_pipeline>dbt debug --profiles-dir .
22:24:48 Running with dbt=1.10.22
22:24:48 dbt version: 1.10.22
22:24:48 python path: C:\Users\pc\anaconda3\envs\ai_env\python.exe
22:24:48 os info: Windows-10-10.0.26200-SP0
22:24:49 Using profiles dir at .
22:24:49 Using profiles.yml file at .\profiles.yml
22:24:49 Using dbt_project.yml file at C:\Users\pc\Desktop\tp_chapitre2_pipeline\dbt_pipeline\dbt_project.yml
22:24:49 adapter type: duckdb
22:24:49 adapter version: 1.10.0
22:24:49 Configuration:
22:24:49   profiles.yml file [OK found and valid]
22:24:49   dbt_project.yml file [OK found and valid]
22:24:49 Required dependencies:
22:24:49   - git [OK found]
22:24:49 Connection:
22:24:49   database: ventes
22:24:49   schema: main
22:24:49   path: ../ventes.duckdb
22:24:49   config_options: None
22:24:49   extensions: None
22:24:49   settings: {}
22:24:49   external_root: .
22:24:49   use_credential_provider: None
22:24:49   attach: None
22:24:49   filesystems: None
22:24:49   remote: None
22:24:49   plugins: None
22:24:49   disable_transactions: False
22:24:49 Registered adapter: duckdb=1.10.0
22:24:50 Connection test: [OK connection ok]
22:24:50 All checks passed!

```

FIGURE 14 – Exécution de dbt debug.

```

(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline\dbt_pipeline>dbt run --profiles-dir .
22:26:50 Running with dbt=1.10.22
22:26:50 Registered adapter: duckdb=1.10.0
22:26:51 Unable to do partial parsing because saved manifest not found. Starting full parse.
22:26:53 Found 2 models, 456 macros
22:26:53 Concurrency: 1 threads (target='dev')
22:26:53 1 of 2 START sql table model main.ventes_clean ..... [RUN]
22:26:54 1 of 2 OK created sql table model main.ventes_clean ..... [OK in 0.20s]
22:26:54 2 of 2 START sql table model main.ventes_resume ..... [RUN]
22:26:54 2 of 2 OK created sql table model main.ventes_resume ..... [OK in 0.14s]
22:26:54 Finished running 2 table models in 0 hours 0 minutes and 0.88 seconds (0.88s).
22:26:54 Completed successfully
22:26:54 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>

```

FIGURE 15 – Exécution de dbt run.

```

(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>cd ..
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>git add dbt_pipeline
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>git commit -m "Ajout des modèles dbt"
[main c355d06] Ajout des modèles dbt
6 files changed, 324 insertions(+)
create mode 100644 dbt_pipeline/.user.yml
create mode 100644 dbt_pipeline/dbt_project.yml
create mode 100644 dbt_pipeline/logs/dbt.log
create mode 100644 dbt_pipeline/models/ventes_clean.sql
create mode 100644 dbt_pipeline/models/ventes_resume.sql
create mode 100644 dbt_pipeline/profiles.yml

```

FIGURE 16 – Commit des fichiers dbt.

Partie 6 : Tests de Qualité avec dbt

```
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>cd dbt_pipeline

(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>dbt test --profiles-dir .
22:30:05 Running with dbt=1.10.22
22:30:05 Registered adapter: duckdb=1.10.0
22:30:06 Found 2 models, 3 data tests, 456 macros
22:30:06
22:30:06 Concurrency: 1 threads (target='dev')
22:30:06
22:30:06 1 of 3 START test not_null_ventes_clean_produit ..... [RUN]
22:30:06 1 of 3 PASS not_null_ventes_clean_produit ..... [PASS in 0.05s]
22:30:06 2 of 3 START test not_null_ventes_clean_quantite ..... [RUN]
22:30:06 2 of 3 PASS not_null_ventes_clean_quantite ..... [PASS in 0.02s]
22:30:06 3 of 3 START test not_null_ventes_resume_categorie ..... [RUN]
22:30:06 3 of 3 PASS not_null_ventes_resume_categorie ..... [PASS in 0.02s]
22:30:06
22:30:06 Finished running 3 data tests in 0 hours 0 minutes and 0.39 seconds (0.39s).
22:30:06
22:30:06 Completed successfully
22:30:06
22:30:06 Done. PASS=3 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=3
```

FIGURE 17 – Exécution des tests dbt (dbt test).

```
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline\dbt_pipeline>cd ..

(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>git add dbt_pipeline\models\schema.yml

(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>git commit -m "Ajout des tests simples dbt"
[main 34823bc] Ajout des tests simples dbt
1 file changed, 18 insertions(+)
 create mode 100644 dbt_pipeline/models/schema.yml

(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>
```

FIGURE 18 – Commit après l'ajout du fichier schema.yml.

Partie 7 : Orchestration avec Dagster

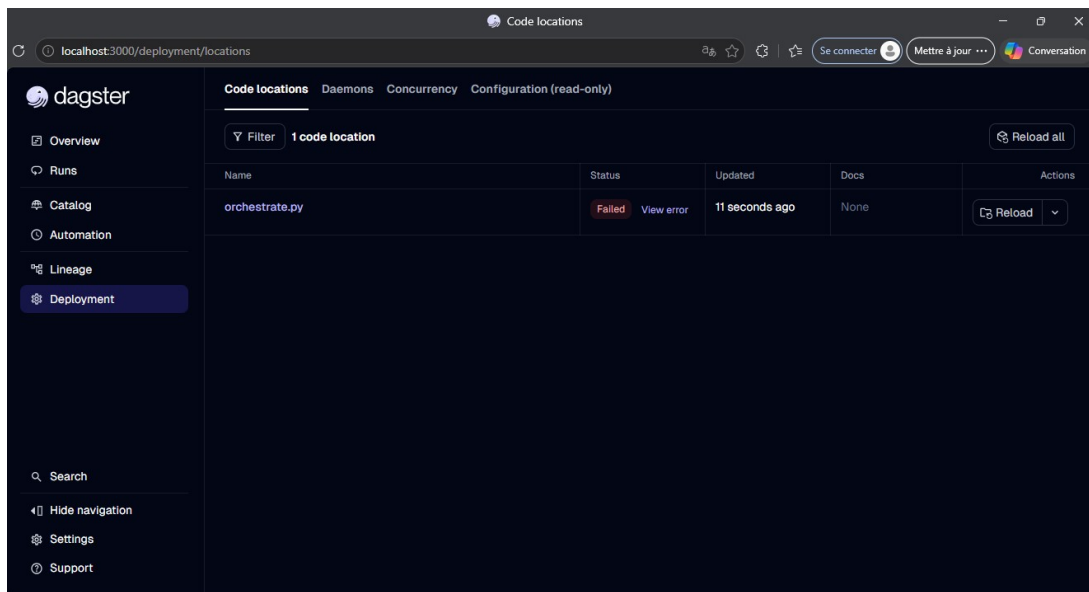


FIGURE 19 – Interface Dagster montrant le pipeline.

```
C:\Users\pc\Desktop\tp_chapitre2_pipeline>git add pipeline\orchestrate.py
C:\Users\pc\Desktop\tp_chapitre2_pipeline>git commit -m "Ajout de l'orchestration Dagster"
[main 3d2a1c8] Ajout de l'orchestration Dagster
1 file changed, 22 insertions(+)
create mode 100644 pipeline/orchestrate.py
```

FIGURE 20 – Commit de l'orchestration Dagster.

Partie 8 : Intégration Continue (CI)

```
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>git add .github  
  
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>git commit -m "Ajout d'une CI simple"  
[main 687ba7f] Ajout d'une CI simple  
1 file changed, 27 insertions(+)  
create mode 100644 .github/workflows/ci.yml  
  
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>
```

FIGURE 21 – Commit de la configuration CI GitHub Actions.

```
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>git log --oneline  
687ba7f (HEAD -> main) Ajout d'une CI simple  
3d2a1c8 Ajout de l'orchestration Dagster  
34823bc Ajout des tests simples dbt  
c355d06 Ajout des modèles dbt  
2b2224a Ajout de l'étape de validation minimale  
1dd0b8d Ajout de l'étape d'ingestion  
25df440 Initialisation du projet pipeline  
  
(ai_env) C:\Users\pc\Desktop\tp_chapitre2_pipeline>
```

FIGURE 22 – Historique des commits Git (git log).

===== RÉPONSES AUX QUESTIONS =====

Réponses aux Questions

1. **En quoi ce pipeline est plus structuré qu'un notebook ?**

Le pipeline sépare clairement les étapes (ingestion, validation, transformation, orchestration). Il est modulaire, reproductible, versionné et automatisable, contrairement à un notebook monolithique et difficile à maintenir.

2. **Quel est le rôle de Git ?**

Git permet de versionner le code, suivre l'historique des modifications, faciliter la collaboration et revenir à des versions stables.

3. **Quel est le rôle de requirements.txt ?**

Il liste toutes les dépendances Python nécessaires, permettant de reproduire exactement le même environnement sur une autre machine.

4. **Pourquoi ajouter validate.py ?**

Il permet de vérifier la qualité et la structure des données avant toute transformation, évitant ainsi la propagation d'erreurs dans le pipeline.

5. **Quel est le rôle de dbt test ?**

dbt test exécute automatiquement des tests de qualité de données (not_null, etc.) définis dans schema.yml.

6. **Qu'apporte Dagster ?**

Dagster permet d'orchestrer l'exécution des différentes étapes dans le bon ordre, avec suivi visuel et gestion des dépendances.

7. **Qu'apporte la CI ?**

La CI automatise les vérifications du code à chaque push (syntaxe, dépendances), permettant de détecter rapidement les erreurs.

8. **Que manque-t-il pour un pipeline pleinement industrialisé ?**

Il manque : le monitoring en production, la scalabilité, le déploiement sur le Cloud, la gestion avancée des erreurs, la sécurité des données, et l'ordonnancement temporel (scheduling).

Conclusion

Ce TP nous a permis de mettre en pratique les principes fondamentaux du **DataOps** en construisant un pipeline complet, modulaire et versionné. Les outils utilisés (dlt, dbt, Dagster, Git) constituent une base solide pour des pipelines de données plus complexes en environnement industriel.